

OPERATING SYSTEMS AND COMPUTER NETWORKS

PROCESS MANAGEMENT

Lecturer: Milos Antonijevic

PROCESSES

DEFINITION:

Process is a running program or, in other words, a program currently in execution.

- Processes begin life as executable object code, which is machine-runnable code in an executable format that the kernel understands.
 - By virtualizing the CPU, i.e. by running one process, then stopping it and running another, the OS gives the illusion that many virtual CPUs exist when in fact there is only one physical CPU (or a few).
 - CPU API includes:
 - Creation of the new process
 - Destruction of the process
 - Waiting for the process to stop
 - Getting process status
- Process can be in one of three states:
- **Running:** A process is running on a processor.
 - **Ready:** A process is ready to run, but for some reason the OS has chosen not to run it at this given moment.
 - **Blocked:** In the blocked state, a process has performed some kind of operation that makes it not ready to run until some other event takes place.

PROCESSES

- To view the current process list on Linux, you can use **ps** (e.g. with "auxf" arguments) command.
- Each process is identified by a unique positive integer called the process ID (pid). The pid of the first process is 1, and each subsequent process receives a new, unique pid.
- Each process consists of one or more threads of execution. The threads are multiple strains of execution within a single process.
- In Linux, processes are organized in the process tree. The process tree is rooted at the first process, known as the init process.
- New processes are created via the `fork( )` system call. This system call creates a duplicate of the calling process. The original process is called the parent; the new process is called the child.
- Every process except the first has a parent.

PROCESSES

Command/key stroke	Effect
CTRL-C	terminate process currently connected to stdin
&	run program in background; stdin stream is disconnected
CTRL-Z	suspend process currently connected to stdin
bg	restart suspended process; stdin still disconnected
fg	reconnect suspended/background process to stdin
kill #	terminate the given process ID

```
main ()  
{  
    while(1);  
}
```

PROCESS MANAGEMENT SYSTEM CALLS

- The **fork()** system call creates a clone of the currently running program. Both the original program and the clone (child) continue execution with the next line of code after the fork() function call. The order of the processes (parent and child) is not deterministic.

```
#include <stdio.h>

main() {
    int i;
    printf("Ready to fork...\n");
    i=fork();
    printf("Fork returned %d\n", i);
    while(1);
}
```

[CTRL+Z] (then try killing parent process)

PROCESS MANAGEMENT SYSTEM CALLS

- The **wait()** system call forces the parent to wait for a child process to finish before continuing its execution. This ensures predictable order of process execution.

...

```
else {  
    // parent process (original process) execution  
    int wc = wait(NULL);  
    printf("hello, I am parent of %d (wc:%d) (pid:%d)\n", rc, wc, (int) getpid());  
}
```

- The child process will always run first. [[Why?](#)]

PROCESS MANAGEMENT SYSTEM CALLS

- The **exec()** family of system calls (exec, execl, execlp, execl, execv, execvp, execvpe) are used when there is a need to run a program that is different from the calling program.
- exec() system call loads the code from the executable (external program binary) and overwrites its current code segment with it. Then the OS runs that code. In other words, exec() does not create a new process, but transforms currently running program into a different program.

...

```
} else if (rc == 0) {  
    // child (new process)  
    printf("hello, I am child (pid:%d)\n", (int) getpid());  
    char *myargs[3];  
    myargs[0] = "wc";  
    myargs[1] = "test.txt";  
    myargs[2] = NULL; // marks end of array  
    execvp(myargs[0], myargs);  
    // execution of the child code ends here!  
}
```

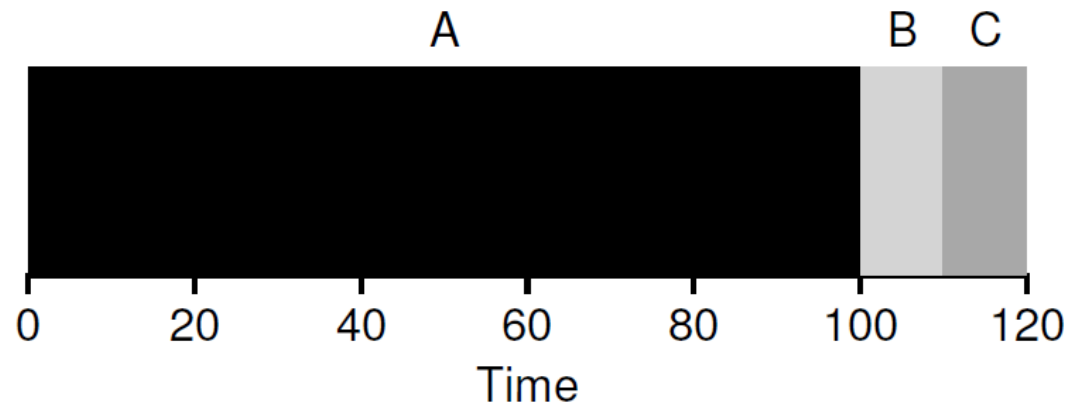
PROCESS MANAGEMENT C LIBRARY

- The **system()** function is a C standard library function that puts the fork(), exec(), and wait() functions together into a single call...

```
#include <stdio.h>
#include <stdlib.h>
main() {
    printf("Ready to system()...\n");
    system("wc test.txt");
    printf("Original process continued?\n");
    sleep(4);
    printf("Yes\n");
}
```

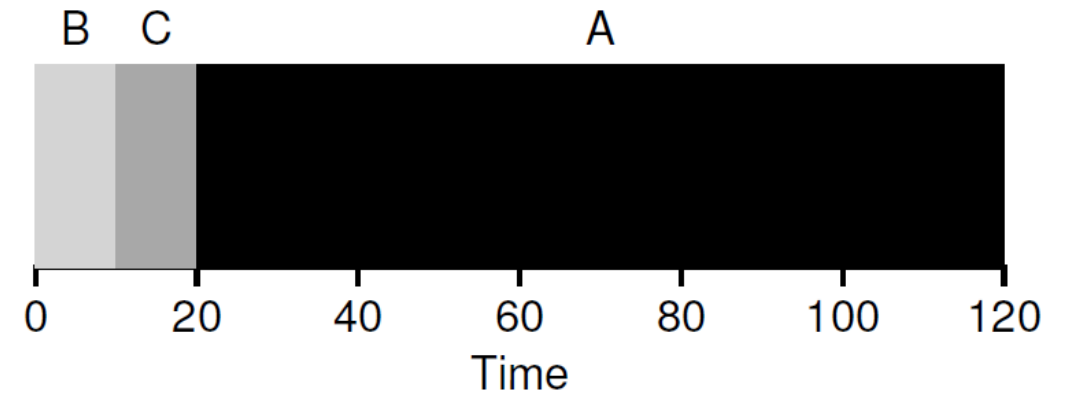
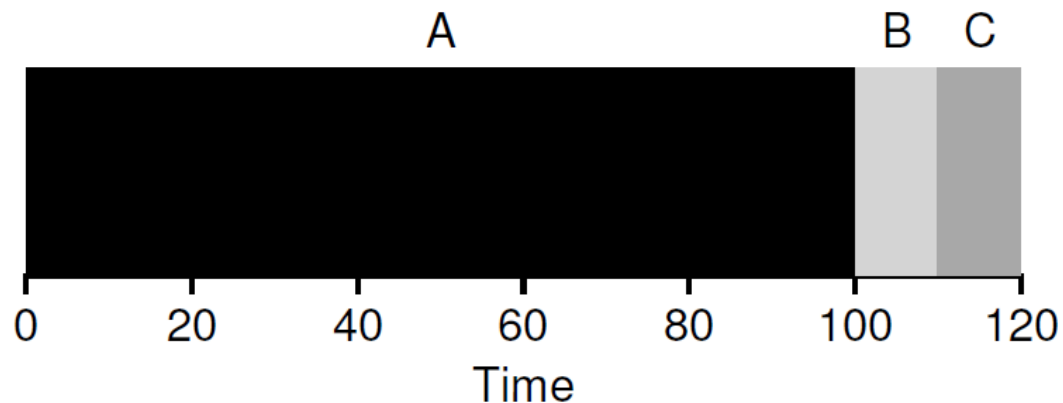
PROCESS SCHEDULING - FIFO

- There are different policies which OS can implement in order to deal with multiple processes that come to execution (almost) at the same time.
- **FIFO** (First in – first out) is the most basic implementation of scheduling. The jobs (processes) are run in the order in which they arrive, and (if we assume that there is only one CPU) each process is waiting until the previous is finished its execution.
- The main problem with FIFO is so called convoy effect, where several relatively short consumers of a resource get queued behind a heavyweight resource consumer.



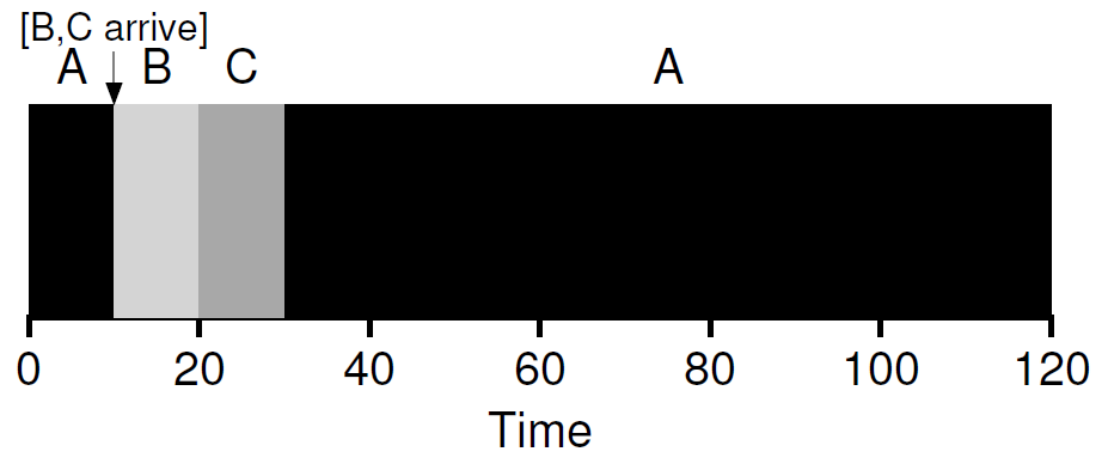
PROCESS SCHEDULING - SJF

- **SJF** (Shortest Job First) runs the shortest job first, then the next shortest, and so on.
- Comparison of average turnaround time of FIFO and SJF:
- **tFIFO** = $(100+110+120)/3 = 110$ **tSJF** = $(10+20+120)/3 = 50$
- Problem with SJF is if jobs does not arrive the same time, the following jobs will still wait for the completion of previous jobs (like FIFO).



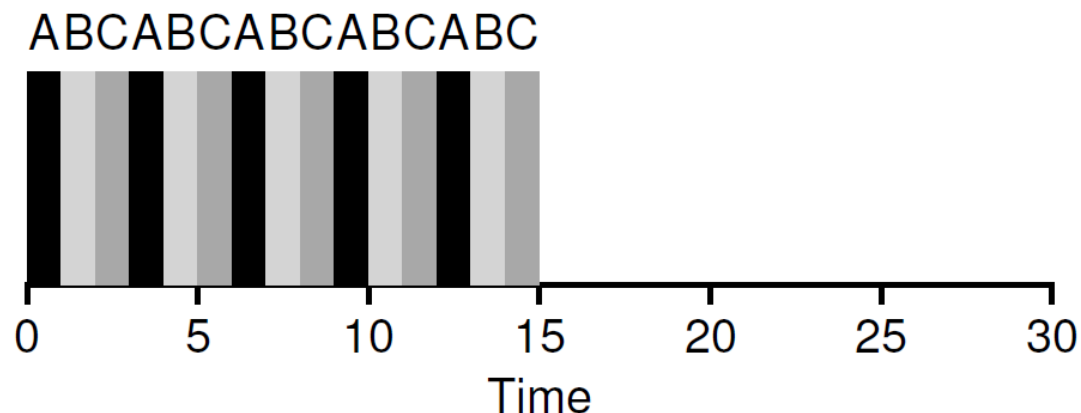
PROCESS SCHEDULING - PSJF

- **Preemptive Shortest Job First** (PSJF) scheduler at the moment the new job enters the system, determines which of the remaining jobs (including the new job) has the least time left, and schedules that one.
- In this case, PSJF would preempt A and run B and C to completion and when they are finished the A's remaining time would be scheduled.



PROCESS SCHEDULING - RR

- **Round Robin (RR)** does not run the jobs to completion but runs a job for a "time slice" and then switches to the next job in the run queue. It repeatedly does so until the jobs are finished.
- The main challenge is to determine the right length of time slice period.
- The response times of jobs are better with RR, but turnaround times are worse. What RR is doing is stretching out each job as long as it can, by only running each job for a short bit before moving to the next. In this case, A will finish in 13 seconds, B in 14, and C in 15 seconds.



LOTTERY SCHEDULING

- **Proportional-share** is based on the model that guarantee that each job obtain a certain percentage of CPU time.
- In order to determine which process should get next to run, OS holds a lottery. Processes that should run more often should be given more chances to win the lottery.
- In order for this to be accomplished, the concept of **tickets** is introduced. The percent of tickets that a process has represents its share of the system resource in question. For example, if process A has 75 tickets and process B has 25 tickets, process A will receive 75% of the CPU and B the remaining 25%.
- The scheduler know how many total tickets are and then picks a random winning ticket, which is a number between 0 and total-1. The process that owns a winning ticket is executed.

LOTTERY SCHEDULING

USER A has 100 assigned tickets by the OS

USER B has 100 assigned tickets by the OS

Within his user domain, USER A assigns 500 tickets to process A1, and 125 tickets to process A2.

Within his user domain, USER B assigns 10 tickets to process B1.

How should OS deal with assigning tickets on system level in this case?

STRIDE SCHEDULING

- Lottery Scheduling is probabilistic model which provides higher fairness of system share the longer time of jobs execution is.
- **Stride scheduling** is deterministic scheduling model that delivers exact proportions of system usage.
- Each job in the system has a **stride**, which is inverse in proportion to the number of tickets it has. For example, with jobs A, B, and C, with 100, 50, and 250 tickets, we can calculate the stride of each job by dividing some large number by the number of tickets each process has been assigned (if we divide 10,000 by each of those ticket values, we obtain the following stride values for A, B, and C: 100, 200, and 40).
- Every time a process runs, we will increment a counter for it (called **pass** value). The system then runs the process with the lowest pass value so far.

LINUX COMPLETELY FAIR SCHEDULER (CFS)

- In order to determine how long (time slice) one process should run before considering a switch to another process, CFS divides a predefined value (sched_latency) by the number of processes.
- CFS then schedules the first job and runs it until it has used “time slice” period of (virtual) runtime, and then checks to see if there is a job with lower vruntime to run instead. In case there is, CFS would switch to running that job.
- In order to prevent time slice to have a very low value (which would increase total time for switching between jobs), CFS uses a minimal predefined value for time slice (min_granularity).
- CFS also enables controls over process priority, enabling users or administrators to give some processes a higher share of the CPU. In Linux, this parameter can be set anywhere from -20 to +19 for a process, with a default of 0. Positive values imply lower priority and negative values imply higher priority.